Skills Reference

Fifteen Agent Skills encoding a phase based engineering workflow, from vague idea to shipped, documented code. Nine are upstream from jsmastery-pro/skills; six are additions in this fork, marked **fork**.

There is no mandated playbook. Run whichever skills a change needs, in whatever order fits. `/scope` recommends a workflow tier, but it is a default you override per feature.

The model in one page

Three rules the whole suite obeys.

1. **The engineer decides; the AI recommends.** Any AI initiated check is offered, never run on your behalf. What it finds is surfaced for you to decide, never silently resolved.
2. **Suggestions, never gates.** Every step after `/develop` is skippable, and `done` is yours to declare. A skipped step is recorded as skipped, not as incomplete.
3. **Every question carries exactly one recommended option** with a one line why. Never a neutral menu.

Workflow tiers — one rigor dial, set per project and overridable per feature. It decides which boxes a feature carries and therefore which skill closes it.

Tier	Stages after `/develop`	Closed by
`Prototype`	nothing, rely on `/develop`'s own build time self check	`/develop`
`Alpha`	`/check verify`	`/check verify`
`Beta`	`/check verify` then `/test`	`/test`
`GA`	adds `/check review` then `/document`	`/test`, then review and docs

The usual loop. `/scope` (what) → `/architect` (decide) → `/develop` (build) → `/check verify` → `/test` → `/check review` → `/document` → `/sync` (record). Bootstrap once with `/audit`. The fork additions run on their own cadence, off the loop.

State lives in files, not in the chat. Every skill reads from disk and writes back, so a fresh session loses nothing. Skills suggest `/clear` at handoffs to keep long sessions cheap.

Planning

`/scope` — what to build

Does. Turns a product idea into a living, coarse scope in `docs/scope/` and keeps it current. Seeds *what*; never designs or builds.

How. Four entries: `plan` (new product), `replan` (next slice), `add <feature>` (enroll one named feature), or bare `/scope` to reconcile after shipping and queue what is next. All three writing modes edit in place, never a new dated file. Picks a **build approach** that shapes the slices: `skateboard` (thinnest usable whole), `tracer-bullet` (thin end to end path), `facade` (UI first on placeholders), `journey` (full user journey per phase). Sets the project's workflow tier. Each feature becomes a row plus a section with boxes: `Design it` → `Build it` (2 to 5 milestones) → `Verify it` → `Test it`, exactly the boxes its tier warrants.

Owns. `docs/scope/`. Writes no specs, no code, no `AGENTS.md`.

Example. `/scope plan` on a new habit tracker. It interviews you, recommends `skateboard`, proposes `Alpha`, and writes a scope with six features. Feature 3 "streak calculation" is marked `Needs spec? yes`, because how a streak survives a missed day is a decision nobody has made.

`/wayfinder` — chart a foggy effort fork

Does. For an effort too big and too foggy for one session: the destination is nameable, but you cannot yet state every question between you and it. Plans decisions; never builds.

How. Two modes. `chart` writes a **map** (destination plus decisions so far) and a folder of **tickets** under `docs/wayfinding/`, each holding one question. `work` resolves them **one per session** — resolving a ticket clears fog ahead of it, which turns whatever just became sayable into fresh tickets. Ticket types include `interview` and `prototype`, which only resolve through live exchange or real code. Deliberately does not pre slice fog into tickets. Done when nothing is left to decide, then hands off to `/scope` or `/architect`.

Owns. `docs/wayfinding/`. Recommends the next command and stops; never runs it.

When it is the wrong skill. Ordinary not yet decided is not fog — use `/scope`. Fog is a question you cannot state precisely because it hangs on something else unsettled.

Example. "Move billing off Stripe." You cannot even list the questions. `/wayfinder` charts a map with four tickets; resolving "what does our data model actually assume about Stripe?" reveals two more nobody could have named on day one.

`/architect` — decide, and write it down

Does. Runs when a load bearing technical decision is unmade: choosing between approaches, designing a feature or page, picking a stack, or when `/develop` says a decision is owed. Asks deep questions, recommends an answer, writes a build spec.

How. Four modes by decision type: `ARCHITECTURE` (new stack or foundation, no code to read), `ENHANCEMENT` (improve or scale something existing), `CROSS-CUTTING` (standardise a pattern codebase wide, e.g. error handling, logging, naming), `FEATURE` (one feature's design). Subagents only read, fetch, or cross check; the main thread writes. Offers a **cross model spec check** — a different model reads the finished spec for gaps. Whatever it finds is surfaced, never auto applied. Supporting evidence goes in the spec's `rationale.md`, never in the scope folder.

Owns. `docs/specs/`, exclusively. It is also the only skill that may clear an `Assumed` spec.

Example. `/architect streak calculation`. It asks what a streak means across timezones, recommends storing the user's local date at write time with a one line why, and writes `docs/specs/0003-streaks/` with acceptance criteria, a value sourcing table, and a build plan.

Building

`/develop` — build it

Does. Builds a feature, UI or backend, from an approved design: a page, component, API, service, or data slice.

How. Gates first, then acts. Step 0 enumerates every value the build must produce or display; any required value with no named source is an *owed decision*, and the run stops and routes you to `/architect`. Otherwise it reads the spec plus `AGENTS.md` and builds. Two tracks: a **UI track** (routes on what design source you have — Figma

MCP, screenshots, an existing `design.md`, or no design at all) and a **logical track**. Asks only what the design left open. Advances the scope: milestone boxes, `Build it`, feature status. Reads `AGENTS.md`'s `## Git` setting to decide whether to branch and commit.

Owns. App code, CSS and tokens, `design.md`, and its own scope box touches. One narrow exception: it may create a spec in `Status: Assumed` when you choose to build now and record the assumption — the assumption record only, never the rationale.

Example. `/develop streak calculation` with the spec in place. It builds the module, ticks two milestones, sets the feature `in-progress`, emits a `verify.md` checklist, and asks where to save it.

`/debug` — find the root cause

Does. Finds and fixes the root cause of one bug: something failing, throwing, or behaving wrong.

How. A reproduce → localize → hypothesize → test → fix → verify loop. Step 1 sets a strict bar: the reproduction must be **red capable, deterministic or pinned to a high rate, and already run once** before anything else proceeds, with an explicit escalation protocol for flaky bugs. Makes the **minimal** fix. No features, no unrelated refactors. Has an **off ramp**: if this is not one bug, it routes to `/recover`; if the bug reveals a flawed decision, to `/architect`.

Owns. The minimal code fix. Hands the regression test to `/test`, or writes a failing then passing test inline when that is the fastest proof.

Example. A streak resets at midnight UTC for a user in Tokyo. `/debug` reproduces it by pinning the clock, localizes to a `toISOString()` call, fixes that one line, and confirms the reproduction now passes.

Verifying

`/check` — prove it, then review it

Does. Two independent modes, run before merge.

How. `verify` drives the **real application** and proves behavior against the spec: every acceptance criterion met, every specced surface actually built and live. It never says PASS if it did not run the app. It distinguishes *specced but missing* (never built) from *specced but not applied* (built but not live, e.g. a generated migration nobody ran). `review` runs a senior code review **on a different model than wrote the code**, which is the point: a model rarely finds its own blind spots. Findings go to `docs/reviews/`.

Owns. `docs/reviews/`. Never edits your code — it reports.

Example. `/check verify streaks` opens the app, exercises five acceptance criteria, finds AC-4 fails because the migration was generated but never applied, and reports FAIL with the evidence path.

`/test` — write the suite

Does. Writes a test suite for code you just built or changed.

How. Targets **uncommitted changes** automatically; the git working tree defines the scope, so there is no scope question. Reads `test-preferences.json` for your framework and conventions, asking and saving it once if absent (`setup` mode). Picks a strategy per file: happy path, edge cases, error states, accessibility. Always asks whether to run the suite after writing. At `Beta`/`GA` it is the closer: on green it offers `done`, never gates it.

Owns. Test files and `test-preferences.json`.

Example. `/test` after the streak fix. It detects Vitest from preferences, writes eight cases including the Tokyo midnight boundary, runs them, and offers to mark the feature `done`.

Recording

`/document` — the human facing prose

Does. Writes the prose a person reads about a change, drafted from the real commits and diff.

How. Four types: `pr`, `changelog`, `release-note`, `postmortem` — pass one or let it ask. Reads the actual diff and recent spec paths for the *why*. Opening or updating a PR is an outward action, so it always shows you the body and confirms before running `gh`, regardless of the git setting.

Owns. PR text, `CHANGELOG.md`, `docs/releases/`, `docs/postmortems/`. No code, tests, or specs.

Example. `/document pr`. It reads eleven commits and the streak spec, drafts a PR body explaining the timezone decision and its tradeoff, shows it to you, and opens the PR only on your go.

`/audit` — bootstrap the AI context

Does. Writes the `AGENTS.md` files every later skill and every AI tool reads. This is the context bootstrapper, run once early and occasionally after.

How. Phases by situation: **greenfield** (no code — asks coding standards, seeds root), **whole repo** (code but no `AGENTS.md` — scans and writes root plus nested area docs), **area** (`/audit src/auth`), **gap fill** (root exists — finds what is true but unrecorded). `AGENTS.md` is canonical and tool agnostic; `CLAUDE.md` is a thin pointer that imports it, never a duplicate. Never overwrites curated content: gaps are collected and applied only with permission. Root stays under ~60 lines; area detail goes in nested docs. Offers architecture presets (clean, DDD, functional, SOLID).

Owns. `AGENTS.md` (root and nested) and `CLAUDE.md` pointers.

Example. `/audit` on an existing repo with no docs. It scans, writes a root `AGENTS.md` with the real stack and daily commands, adds `src/payments/AGENTS.md` for that area's gotchas, and lists three root gaps for you to approve.

`/sync` — keep durable knowledge current

Does. The last step after a change is complete, around merge. Updates `AGENTS.md`, reconciles the scope from repo evidence, and flags specs the change made stale.

How. Surgical edits only — it adds lines and rewrites single lines it owns, never a whole section and never curated prose. Acts as the universal sub task reconciler: it ticks any scope sub task it can verify from repo evidence, sweeping the boxes other skills leave. **Stops immediately if only docs, tests, or lockfiles changed** — no source diff, nothing to sync. It will not create or restructure the root `AGENTS.md`; that is `/audit`'s job, and it flags rather than doing it.

Owns. Exactly what its Boundaries table grants.

Example. `/sync` after merging streaks. It adds one line about the timezone convention to `src/streaks/AGENTS.md`, ticks the `Test it` box it can prove from the new test files, and flags spec 0002 as stale because the change contradicts it.

`/overview` — what this project *is* fork

Does. Answers the broadest question no other file holds: what is this project, for someone with no prior context.

How. Three modes. `update` (the default, run often) keeps `docs/overview.md` current as a structured reference — what it does, who for, how built, the moving parts, and the reasoning behind its shape. `story` writes `docs/project-story.md`, a plain prose telling for a person: no headings, no tables. `check` reports where the document has drifted from what scope and specs now show. Never invents a *why*.

Owns. `docs/overview.md`, `docs/project-story.md`. Never a source of truth for a decision — that is a spec.

Example. `/overview update` after streaks ship. It adds the streak subsystem to the moving parts, notes the timezone decision with a pointer to spec 0003, and reports what changed.

Session and memory (all fork)

`/checkpoint` — the session residue

Does. Captures what the durable files do not own: a hypothesis ruled out, a standing instruction that is not a decision, an open question you are still turning over.

How. `save` writes three headings in `docs/session-notes.md` — `Open threads`, `Ruled out`, `Standing instructions` — and only what scope, specs, and `AGENTS.md` do not already say. If everything of value already landed, it writes nothing and says so. `restore` reads that file plus a light pass over scope and recent specs, states the picture back, and **waits for you to confirm** before continuing, because a wrong carried over assumption is worse than no memory. Ages out entries that have since found a home. Standing instructions are session scoped only: a rule meant to hold from now on is a reflex.

Owns. Three headings in `docs/session-notes.md`, a file shared by section with `/recover`. It reads the whole file and writes every other section back byte for byte.

Example. `/checkpoint save` ending a session where you ruled out server side date math and told the agent not to touch the payments module this week. Two lines saved. Next session, `/checkpoint restore` reads them back and asks whether they still hold.

`/recover` — diagnose the kind of failure fork

Does. Runs when something has gone wrong and it is not obvious *what kind* of wrong. The instinct is to keep prompting for fixes; this skill diagnoses before responding.

How. Three failure modes, three responses. An **isolated bug** → hands `/debug` a compact brief (observed, expected, reproduction, attempts so far). A **session gone wrong through repeated patching** → hard reset: write a note into `docs/session-notes.md`, end the session, restart with `/checkpoint restore`. A **foundation on a wrong assumption** → rethink: name the assumption, propose the corrected approach, wait for confirmation before touching code. States its diagnosis without asking, but pauses before a reset ends the session or a rethink changes code.

Owns. `## Reset notes` in `docs/session-notes.md`. `/checkpoint` removes them once spent; it may not rewrite them on its own judgment.

When. This is what `AGENTS.md`'s circuit breaker points at: if the same problem persists after one corrective prompt, stop and run `/recover` before trying again.

Example. Third failed patch on the same test. `/recover` diagnoses a polluted session rather than a third bug, writes a reset note naming the two dead ends, and tells you to start fresh.

`/reflex` — turn a correction into a standing rule fork

Does. Captures the rule that exists only because you said it out loud, and would otherwise die at `/clear`.

How. Routes before capturing, and most candidates route away: something mechanical and deterministic belongs in a hook where it is *enforced* rather than remembered; a standard to enforce codebase wide is an `/architect` `CROSS-CUTTING` spec; a fact visible in the code is `AGENTS.md`; something true only this week is `/checkpoint`; a one off that generalizes to nothing is not captured at all. What survives becomes one line, trigger then action, drafted and **confirmed with you before writing** — a wrong rule is read by every later session. `audit` prunes: contradictions, duplicates, stale triggers, and the oldest rules as graduation candidates. Capped at 20 rules, because a file long enough to be skipped stops being read.

Owns. `docs/reflexes.md`. Never edits `AGENTS.md`; the root pointer that makes the file get read is written by `/audit`. Graduation is two steps and you move the line yourself — no skill carries the text across.

Example. You tell the agent, for the third time, to run the migration check before touching a schema file. `/reflex` proposes `- When a schema file changes, run the migration check before proposing the diff. (added 2026 08 19)`, you confirm, and no later session needs telling.

`/imprint` — keep the UI consistent fork

Does. Records what a finished component actually is, precisely enough that the next one can match it. Prevents the slow drift where spacing wanders, a second shade of blue appears, and the app looks built by several people with different taste.

How. `capture` (the default, after building any component) extracts only what matters for consistency — background, border, radius, text colors and sizes, spacing, hover and focus, shadow, accent — and appends to `ui-registry.md`. Deliberately ignores width, height, layout, and positioning as too context dependent. It compares against the baseline first: a deviation is either a deliberate exception, recorded with a note, or a mistake to fix, and it asks which. `audit` scans a whole body of UI at once, finds every conflicting variant, and proposes a baseline for you to confirm — also the right mode for UI freshly exported from a design tool.

Owns. `ui-registry.md`. `design.md` (owned by `/develop`) is the standard and **wins** on disagreement; this skill reports the conflict and never writes it.

Example. `/imprint` after building a settings card. It records radius `8px`, border `border-subtle`, padding `16px`, and flags that the hover state uses a raw hex where the rest of the app uses a token.

Who writes what

Artifact	Owner
`docs/scope/`	`/scope` (also ticked by develop, sync, check verify, test)
`docs/specs/`	`/architect` (develop may create `Assumed` specs only)
App code, CSS tokens, `design.md`	`/develop` (`/debug` writes minimal fixes)
Test files, `test-preferences.json`	`/test`
`docs/reviews/`	`/check review`
PR body, `CHANGELOG.md` , `docs/releases/` , `docs/postmortems/`	`/document`
`AGENTS.md` (root + nested), `CLAUDE.md` pointers	`/audit` , kept current by `/sync`
`docs/overview.md` , `docs/project-story.md`	`/overview` **fork**
`docs/session-notes.md`	shared by section: `/checkpoint` + `/recover` **fork**
`docs/wayfinding/`	`/wayfinder` **fork**
`docs/reflexes.md`	`/reflex` **fork**
`ui-registry.md`	`/imprint` **fork**

If `docs/` is a published documentation site, the `docs/` based artifacts move to `.workflow/` so they do not ship with your site. `ui-registry.md` sits at the repo root and does not move.

Quick reference

You are...	Run
starting a product	`/scope plan`
facing something too foggy to plan	`/wayfinder`
holding an unmade technical decision	`/architect <feature>`
ready to build	`/develop <feature>`
looking at one broken thing	`/debug`
unsure *what kind* of broken	`/recover`
about to merge	`/check verify`, then `/check review`
needing tests	`/test`
writing for humans	`/document pr`
onboarding a repo to the workflow	`/audit`
done with a change	`/sync`
wanting the whole picture	`/overview update`
ending a session with loose threads	`/checkpoint save`
correcting the agent again	`/reflex`
finishing a UI component	`/imprint`